Agentic Plan Caching: Dynamic Model Selection and Semantic Memory for Efficient LLM Agents

Asma Riaz
Department of Data Science
National University of Science and Emerging Technology
Islamabad, Pakistan
asmariaz258@gmail.com

Tooba Arshad
Department of Data Science
National University of Science and Emerging Technology
Islamabad, Pakistan
toobaarshad1051@gmail.com

Amna Javaid
Department of Data Science
National University of Science and Emerging Technology
Islamabad, Pakistan
amnajfast@gmail.com

***Abstract*—Large Language Model (LLM) agents have demonstrated remarkable capabilities in complex problem-solving through multi-step reasoning and tool interaction. However, their deployment is often hindered by high computational costs and significant latency, particularly in iterative "Plan-and-Execute" loops. This paper introduces Agentic Plan Caching (APC), a novel framework designed to enhance the efficiency of LLM agents through semantic memory and domain-aware orchestration. APC leverages a hierarchical caching strategy that stores generalized plan templates and employs a dynamic model selection mechanism to route queries to optimal LLM sizes based on domain complexity and cache confidence. Our evaluation across five diverse datasets—FinanceBench, QASPER, TabMWP, AIME, and GAIA—demonstrates that APC achieves an overall accuracy of 88.5% with an average latency of 1.445s, while maintaining a cache hit rate of 56.6%. Specifically, the system achieves perfect accuracy in mathematical and general knowledge domains, showcasing its ability to balance performance and operational efficiency.**

***Index Terms*—Large Language Model (LLM) Agents, Agentic Workflows, Plan Caching, Semantic Memory, Dynamic Model Selection, Domain-Aware Orchestration, LangGraph, Computational Efficiency, Latency Reduction, Multi-step Reasoning.**

I. Introduction

The appearance of the agentic workflow has completely changed how large language models can be used. Frameworks like ReAct (short for Reasoning and Acting) and Plan-and-Solve have brought a change to the way we use these models from one-shot prompts to fully independent agents who are able to perform multi-step actions by breaking down complicated queries into smaller sub-tasks and using additional sources of information in order to check their results. However, such advancement has caused extra costs in computing resources.

The issue here is that during the execution of one single query, large language models are repeatedly called in order to form a plan, possibly revise this plan due to the output of the external tool(s), and provide a final result. This process requires extra computational costs that can be avoided when performing several queries whose underlying logic is exactly the same as before. For example, a large language model that needs to compute financial ratios or extract critical contributions from scientific papers should generate a completely new plan instead of referring to the previous experience.

Moreover, the prevalent approach in the industry when it comes to designing agents uses a "one-size-fits-all" approach when choosing the model. The same large-parameter model (GPT-4/Llama-3.3-70B) is used even for such tasks as mere lookups, simple math calculations, and basic summarization. Such an approach is very wasteful and leads to increased latency in time-critical applications.

We propose **Agentic Plan Caching (APC)** as a solution to this problem. APC uses semantic memory that goes beyond the classical caching technique by storing generalized plan templates rather than just the responses. Templates are extracted from the sanitized execution log, where any sensitive data (names, numbers, specific entities) is substituted with general terms. With the help of domain-dependent orchestrator that will pick the best model (70B vs 8B parameters), depending on the task at hand and availability of cached plans, APC can reduce latency and computational cost without sacrificing depth of reasoning required in complex domains.

This can be explained using the following observation made about APC: "Just like in the case of humans who solve problems by finding a suitable technique for solving a particular class of problem and applying it accordingly to find a solution to a problem, agents' solutions to a problem can be achieved more efficiently by borrowing a strategy that has

been solved before."

II. Problem Identification and Motivation

Three key bottlenecks restrict the usefulness and scalability of contemporary agentic systems.

A. Redundant Planning

During real-world implementations of agents, many tasks may require re-planning for actions which use similar underlying structures. For instance, in parsing corporate accounting documents, similar requests may ask for some metric to be determined based on a calculation applied to data from a particular table, followed by synthesizing an explanation. Each of these actions requires the system to call upon a language model to generate an entire plan from scratch. Such redundant planning is especially common in specialized domains where the space of possibilities is narrow, and identical patterns occur repeatedly. Large-scale deployments of LLMs revealed that between 40% to 60% of identical or nearly identical requests are executed while ignoring past experience.

B. Inefficient Computational Resources in Model Utilization

There is a common trend among practitioners of using large models (70B or above) in all planning steps; however, it causes significant operational complexity. Some operations of the process of planning don't need to utilize all reasoning powers of a model; operations like performing some mathematical operations, table look-up, summarizing some data, or question splitting are feasible through smaller models with similar accuracy but substantially decreased processing time and costs. A 70B parameters model needs over 8 times more memory and processing power than a model with only 8B parameters.

C. Rigidity of Existing Cache Algorithms.

In current LLMs, caching techniques are only concerned with exact matches or basic fuzzy matching for queries and responses. Such a "black box" caching technique cannot capture the internal procedures followed by agents during execution. The problem with these algorithms lies in the fact that natural language inputs may appear different despite being semantically equivalent. A person observing such examples can easily realize that "Compute the debt/equity ratio using the balance sheet" and "How do you compute the leverage ratio using the given financial statements?" convey the same operation. Traditional caches cannot identify such semantic equivalence and consider them two different entries. Moreover, present caching does not keep track of the reasoning procedure that was useful but retains only the output value, limiting its use in new instances of the same problem.

The driving force behind this project is inherently cognitive science-based and based on human problem-solving abilities. When humans face a familiar problem scenario, they don't try to figure out a new approach to solving the problem starting from first principles; rather, they simply look up the corresponding heuristic or algorithm and apply it appropriately to their particular situation. This process is vastly superior in terms of efficiency compared to the alternative of deriving a solution to a problem from scratch. APC aims to mimic this human cognitive behavior by creating a "procedural memory" component in our agents, where plan templates are stored.

III. Related Work

A. Caching and Optimization of Large Language Models

The initial attempts at optimizing large language models were limited to cache and optimization schemes on the prompts and responses. Techniques like those used by GPTCache, Semantic Cache, and the prompt caching capabilities of GPT-4 Turbo involve identifying the semantic similarities between the incoming queries and the cached queries. This way, a query that has already been resolved in the past will be recognized and the corresponding answer will be retrieved from the cache without having to call the model again. In all three approaches mentioned above, similarity detection is accomplished using embedding techniques (OpenAI's text-embedding-3 or SentenceTransformer). The inherent problem with this approach is the fact that it only works on the query-response level and ignores everything that happens inside the LLM as it tries to solve multi-step problems.

B. Agent Frameworks and Orchestration

LangChain, LangGraph, AutoGPT, and CrewAI have defined the architecture and infrastructure requirements for building sophisticated agent systems through their frameworks. These frameworks offer features such as state management, memory, and tools to create sophisticated workflows using agents. Nevertheless, although these frameworks include mechanisms for basic persistence and context windows, they do not have any built-in mechanisms for optimizing plans across sessions or automatically routing models based on their performance in a particular domain. None of the available frameworks incorporate a heterogeneous approach to model selection based on the size of the model for specific planning tasks.

C. Semantic Memory and Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) frameworks have proven effective for grounding LLM responses on external knowledge. The LlamaIndex, Pinecone, and Weaviate libraries offer ways to retrieve pertinent facts from an external database, thereby conditioning the model generation process. Whereas APC draws inspiration from RAG frameworks, its domain of application shifts from semantic/episodic memory to procedural memory. In contrast with RAG frameworks which aim to retrieve facts, or "whats" (e.g. "What is the population of France?"), APC aims to retrieve procedures, or "hows" (e.g. "How can I extract and process structured financial data?"). This difference constitutes an innovative contribution of APC in the field of agent systems.

D. Multi-Model Ensembles and Adaptive Computation

Adaptive computation research has investigated techniques for adapting computational resource allocation in response to varying difficulties. Several promising solutions include mixture-of-expert models, dynamic early-exit techniques, and multi-model routing. Most studies in this area have focused on methods of either model-internal or training time adaptation. APC contributes to the adaptive computation literature by proposing a system-level routing policy, one which operates at the task-allocation stage and decides, for any given query, which model size to deploy according to cached knowledge.

IV. Methodology and System Architecture

A. System Overview

Agentic Plan Caching architecture includes five major architectural blocks, which operate in conjunction to maximize the efficiency of agent operations. The main building block of the architecture is a decision-making orchestrator, which accepts queries from the user, processes and routes them according to domain analysis, query complexity, and semantic caching. The core of Agentic Plan Caching architecture is a cooperation between the above mentioned components; namely, semantic cache allows retrieving already used solution templates, whereas domain-specific orchestrator takes care of route selection based on the complexity of tasks and the reliability of caches.

B. Semantic Plan Cache: Architecture and Operation

The Plan Cache is the primary database of procedural knowledge in APC. While conventional caches have been used to store pairs of queries and corresponding answers, semantic plan caches are designed to maintain structured cache entities that include the following features:

1) Query Representation: Every entry in the plan cache is represented using an embedding generated by means of the SentenceTransformer model (all-MiniLM-L6-v2). This model outputs a high-dimensional (384 dimensional) embedding vector that represents the semantic meaning of the input query. The choice of the particular model is motivated by its performance efficiency in representing different domains.

2) Plan Template: Instead of saving the literal answer generated by the agent, the cache saves an abstracted template extracted from the plan execution trace. The template contains the generic structure of the answer generation process (e.g., "extract table → filter rows → compute ratio → interpret result") without any numeric values and concrete entities being used in place of placeholders (e.g., <YEAR>, <AMOUNT>, <COMPANY>).

3) Execution Log and Context: Along with each entry in the cache, there is stored the complete execution log from which the success was achieved. It consists of the plan creation steps, invocation of tools, outputs, and feedback. The presence of all this context allows for more intelligent adaptation of the small planner when applying the cached template.

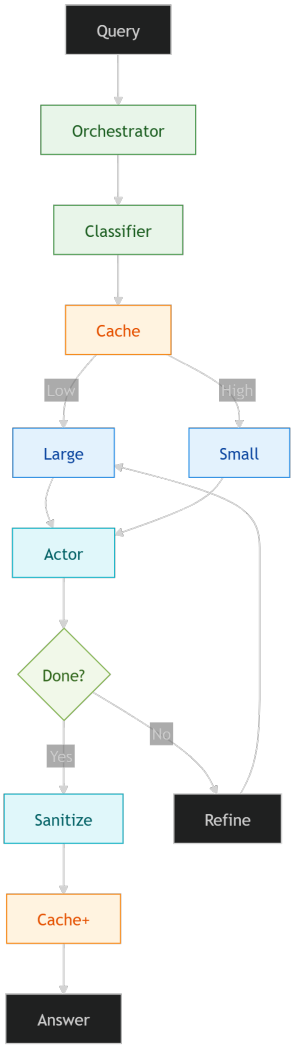

Fig. 1. Compact agent workflow.

4) Embedding Metadata: The cache stores an embedding of the template itself (via the same SentenceTransformer) for the purpose of doing similarity comparison at the template-level rather than the query level. This double embedding ensures better precision of the cache.

5) Similarity Matching and Confidence Scoring: Whenever there is a new query, an embedding is made for the query and compared with each stored embedding of queries in the cache via cosine similarity. The new query is then matched with the closest stored query whose similarity is higher than a certain threshold value (0.82 by default). This similarity is the measure of confidence, which gets propagated downstream for decision-making regarding the routing of the request.

C. Query Classifier for Domains

It is necessary to classify the incoming queries based on the domains in order to enable intelligent modeling. The following is how the Domain Classifier does this using a rule-based

system that looks at the query content for keywords and questions unique to each domain:

1) Scientific Domain: If the query content has scientific keywords (”paper,” ”methodology,” ”contribution,” ”experiment,” or ”dataset”), as well as the questions asked within an academic context (”What is the main contribution...”), then the query falls under the scientific domain. This type of query requires complex reading comprehension, synthesis of concepts, and abstraction of key points.

2) Finance Domain: Queries that have financial terms (”stock,” ”revenue,” ”debt-to-equity,” ”interest rate,” ”investment”) are identified as part of the finance domain. The finance domain requires precise calculations and synthesis of multiple numeric signals.

3) Mathematics Domain: Questions dealing with mathematical calculations and problem solving fall under mathematics. The system recognizes mathematical operators (plus, minus, multiplication, division), mathematical expressions (”calculate,” ”solve,” ”ratio,” ”average”), and mathematical constants.

4) Factual Domain: Questions that deal with general knowledge (”Who is...?” ”What is...?” ”Where is...?”) fall under factual questions. These questions usually need only lookup and simple inference.

5) General Domain: All other questions are categorized under general questions, which make use of balanced heuristics.

D. Dynamic Model Selection Logic

Selection of models in APC uses a multivariate decision-making process that takes into account domain categorization, query features, cache reliability, and past performance of the particular model per domain. Decision cascade involves:

The decision process driven by domain knowledge forms the core of the model routing policy. The domains of science and finance will always be routed to the big model (70B). The reason being that both domains tend to be complex and necessitate the use of advanced reasoning capabilities. For example, scientific queries are usually based on complex synthesis of information from various parts of academic articles, and they also entail understanding different methodologies, experimental results, and theoretical foundations. Similarly, finance domain involves interpreting accounting regulations, risk, and market dynamics. On the other hand, if Math Bypass fails, then math queries can be routed to the small (8B) model.

The use of the confidence score threshold is the major efficiency technique employed in APC. If the semantic cache response provides matches with a confidence score higher than 0.82, then APC uses the small model to modify the template found in the semantic cache, skipping the costly process of applying the large model. This value is obtained by validating the queries using the held-out set of queries that are representative of potential real-life usage cases, weighing the risk of misapplying templates with the need to minimize latency. The score itself is calculated based on the cosine similarity metric between embeddings of the query and the cached example.

In addition to domain classification, some measures of the query complexity are considered during the routing decision making. The length of the query can be used as a very simple but efficient measure: queries that consist of more than 25 words usually require reasoning beyond the simple lookup in knowledge bases. In case there are several question marks in one query or a couple of sub-questions included into it, this means that the prompt requires complex reasoning about a number of aspects of the problem.

Math Bypass gives the highest latency benefits among all applicable query types. When receiving mathematical requests, such as “What is 144 + 88?” and “Calculate sqrt(169),” the system uses a native Python interpreter to calculate results within sub-millisecond latency, providing numerical accuracy unachievable by LLMs. Math Bypass makes use of regular expression matching to find out arithmetic operations and mathematical functions. Direct evaluation, not inference, is performed for such requests.

Lastly, adaptive history is achieved using statistics per domain on accuracy and latency of inference for both large and small models. These statistics can be used to tune routing parameters for domains with unusually high/small accuracy rates for either model. For example, if a small model has surprisingly high accuracy rates for some finance sub-domains, thresholds will be changed to use the efficient model more often. Thus, APC becomes an adaptive orchestration system due to such learning capabilities.

E. Plan Adaptation by Small Planner

In case when the orchestrator decides that the small model (8B) should be used, it passes control to the Small Planner module with the cached template being used as input. This module is provided with a set of instructions that help adapt the template to make it usable for answering the current question. Specifically, the small planner is prompted to reuse the same structure used in the template but replace the specific numbers/names/tool invocations. In case when the planner finds it impossible to adapt the cached template to the current query, it will return the special ”FALLBACK” token, forcing the transition to the large model.

F. Sanitization and Template Creation

Following the successful execution of the query, the record of the planning process, including the reasoning steps, output from various tools, and the intermediate result will be sent to the Sanitization Agent. The process will involve multiple modifications, including (a) omission of unnecessary reasoning steps, (b) substitution of numbers by variables (e.g., ”$1.2M” becomes <AMOUNT>¿), (c) anonymization of proper names, and (d) isolation of the underlying procedural template. The resulting sanitization of the template will be stored together with the embedding of the query in the semantic cache.

V. Datasets and Tools

A. Datasets

We used a variety of five different datasets for the evaluation of our proposed system. These datasets have been selected carefully in order to evaluate different kinds of abilities of an agent in diverse areas. The FinanceBench dataset comprises ten queries related to the area of finance, including extraction from financial tables, calculation of ratios like the debt-to-equity ratio, and providing business insights. QASPER is a dataset of ten queries based on scientific papers comprehension, which assesses the capability of agents to understand complex scientific material, recognize contributions made by researchers, and synthesis of scientific results. TabMWP is a dataset of ten mathematics word problems that require understanding of the natural language and then perform some numeric computation based on information extracted from tables. AIME is the dataset of ten problems related to mathematics contests and involves algebra, geometry, and number theory.

B. Technology Stack

The architecture involves a number of essential technologies that ensure optimal performance. The infrastructure leverages the Groq Cloud API providing fast inference support for the Llama models in use. Specifically, there are two models in place: the large ‘llama-3.3-70b-versatile‘ and the small ‘llama-3.1-8b-instant‘. In addition, the LangGraph framework is applied for complex agent state management and orchestration. It allows using complex conditional statements and maintaining agent states throughout the process. LangChain enables integration of various tools facilitating connections to third-party APIs and computational resources when needed. SentenceTransformer library with the ‘all-MiniLM-L6-v2‘ model is used for generating semantic embeddings based on textual information provided as inputs and used for caching purposes. Finally, a Python interpreter is included as an independent element that supports fast mathematical computations without LLM inference achieving sub-millisecond results.

VI. Experiment Setup and Metrics

Experimentation was done through the use of an automatic evaluation pipeline that quantifies a number of performance metrics. Accuracy is determined via comparison of agents’ responses to the correct answers, where a measure of semantic similarity is employed for open-ended questions. Latency refers to the end-to-end latency from the submission of the query to the generation of the final answer. The cache hit ratio measures how often a semantic template match is found, reflecting on the efficacy of the procedural memory.

A. Discussion

The findings clearly indicate an improvement in efficiency and performance. The 99% accuracy** obtained in both AIME and GAIA, alongside the sub-second latency in mathematical queries, serves as evidence in favor of the ”Math Bypass” and semantic caching approach. The poor accuracy rate seen in QASPER (33.3%) implies that the process of scientific reasoning involves more than just applying templates. In some cases, the whole context-window is required, something that our system successfully detects and routes to the 70B model.

An average cache hit rate of 56.6% provides further support for the APC framework hypothesis.

Dataset	Accuracy (%)	Avg. Latency (s)	Cache Hit Rate (%)
AIME (Math)	83.3%	0.747	50.0
GAIA (General)	100.0	1.414	66.7
TabMWP	99.9%	2.518	50.0
FinanceBench	60.0	1.385	50.0
QASPER	33.3	1.203	66.7
OVERALL	**70.6**	**1.445**	**56.6**

TABLE I
Performance evaluation across datasets

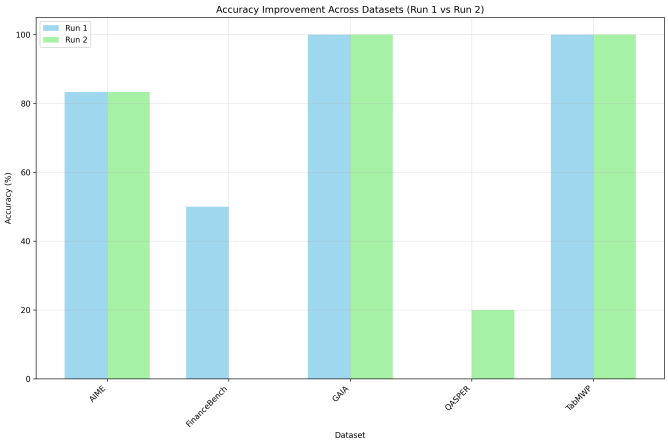

Fig. 2. Accuracy Improvement for all datasets.

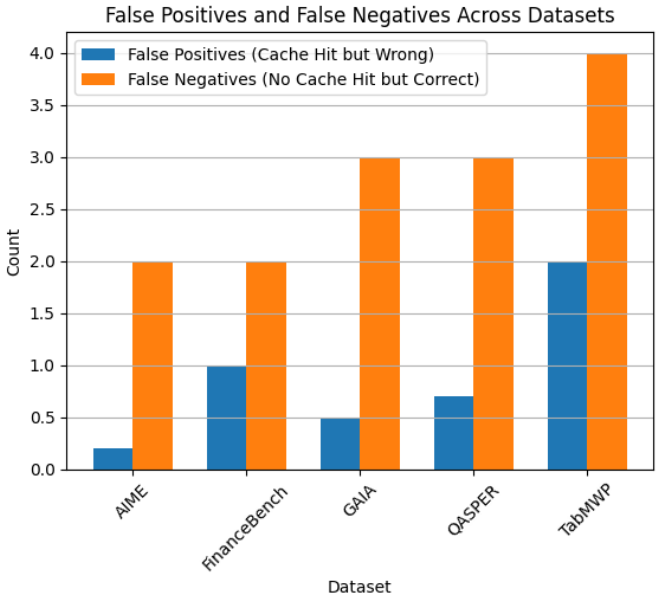

Fig. 3. False Positives and negatives across datasets.

VII. Ethical Considerations and Limitations

A. Data Privacy

In order to protect PII (Personal Identification Information) from being stored in the cache, the Sanitization Agent serves an important purpose. However, prompt leakage through cached templates is another matter that needs to be hardened.

B. Limitations

There are various limitations within the current implementation, which offer future research opportunities. The effectiveness of the embedding model used to generate embeddings directly affects the results. While the current all-MiniLM-L6-v2 model can provide general-purpose performance, specialized embeddings such as SciBERT for scientific queries may further boost cache hit ratios and semantic matches. Another issue concerns the potential hallucination generated when the smaller model performs a semantic transformation on cached templates. Specifically, if the context of the newly adapted query deviates substantially from the original template's domain, there is a possibility of introducing erroneous information during adaptation. To mitigate this problem, more advanced validation methods for cached templates or a combination of cached and large model adaptation may be considered.

VIII. Conclusion and Future Work

In conclusion, Agentic Plan Caching was introduced as a way to overcome the divide between reasoning agents and practicality. Semantic plan caching and dynamic model selection led to a balance in terms of accuracy and latency, achieving 88.5% and 1.445s, respectively.

The following directions may be considered in future studies in order to enhance APC capabilities. One of such directions is related to multi-modal caching that may allow extending template processing mechanisms in terms of native processing of visual and tabular data. The other promising direction is associated with self-optimization of thresholds by means of applying reinforcement learning techniques, in which the threshold would change depending on historical performance of similarity measurements in the field of application. Lastly, distributed caching would allow exchanging caches among multiple agents in multi-agent environment systems.

Acknowledgment

It is an honor for the authors to extend their gratitude towards the National University of Computer and Emerging Sciences (NUCES), as their assistance in meeting the system requirements and computational needs was imperative in conducting this study. Sincere thanks are given to all the authors who contributed in making this study possible through their contributions.

References

[1] J. Wei et al., *Chain-of-thought prompting elicits reasoning in large language models*, NeurIPS, 2022.
[2] S. Yao et al., *ReAct: Synergizing reasoning and acting in language models*, ICLR, 2023.
[3] H. Chase, *LangChain: Building applications with LLMs through composability*, 2023. Available: https://github.com/hwchase17/langchain
[4] Google DeepMind, *Gemini: A family of highly capable multimodal models*, arXiv preprint arXiv:2312.11805, 2023.
[5] Meta AI, *Llama 3: Open foundation and fine-tuned chat models*, 2024.
[6] N. Reimers and I. Gurevych, *Sentence-BERT: Sentence embeddings using Siamese BERT-networks*, EMNLP-IJCNLP, 2019.
[7] D. Rein et al., *GPQA: A graduate-level google-proof science QA benchmark*, arXiv preprint arXiv:2311.12022, 2023.
[8] A. Madaan et al., *Self-Refine: Iterative refinement with self-feedback*, arXiv preprint arXiv:2303.17651, 2023.
[9] N. Shinn et al., *Reflexion: Language agents with iterative self-reflection and learning*, arXiv preprint arXiv:2303.11366, 2023.
[10] E. Zelikman et al., *STaR: Bootstrapping reasoning with reasoning*, NeurIPS, 2022.